0. RESUMEN

Este Trabajo de Fin de Grado se enmarca dentro de un proyecto de investigación en colaboración con el departamento de neurología del Hospital Universitario de Burgos. El objetivo principal es el desarrollo de una aplicación móvil nativa para Android, diseñada para facilitar la telerrehabilitación asíncrona entre terapeutas y pacientes. La solución propuesta implementa una arquitectura cliente-servidor que permite a los profesionales sanitarios diseñar rutinas de ejercicios personalizadas y evaluar el desempeño de los pacientes de forma remota.

Desde el punto de vista técnico, la aplicación ha sido desarrollada utilizando el lenguaje Kotlin y el framework de interfaz declarativa Jetpack Compose, siguiendo una arquitectura MVVM (Model-View-ViewModel) para garantizar la escalabilidad y mantenibilidad del código.

Un aspecto crítico del proyecto ha sido el diseño de la experiencia de usuario (UX/UI), adaptando la interfaz a las limitaciones motoras de los pacientes mediante el uso de comandos simplificados y flujos guiados. Asimismo, se ha implementado un módulo robusto de captura de vídeo utilizando la librería CameraX, resolviendo desafíos técnicos complejos relacionados con el ciclo de vida de la cámara y la gestión de archivos locales. Esta captura de vídeo de alta calidad es fundamental, ya que constituirá la base de datos necesaria para futuros módulos de Inteligencia Artificial destinados a la detección de poses y la corrección automática de ejercicios.

El resultado final es una herramienta funcional que permite gestionar roles de usuario (administrador, terapeuta, paciente), realizar y grabar sesiones de rehabilitación, y visualizar la evolución del paciente mediante gráficas de rendimiento, sentando las bases tecnológicas para un sistema de telerrehabilitación inteligente y autónomo.

1. Introducción

La enfermedad de Parkinson es uno de los retos de salud más importantes hoy en día, siendo el segundo trastorno neurodegenerativo más común. Se caracteriza por síntomas que dificultan el movimiento, como rigidez, temblores y lentitud. Aunque la medicación es necesaria, está demostrado que la rehabilitación física constante es la clave real para que los pacientes mantengan su independencia el mayor tiempo posible.

Sin embargo, aquí nos encontramos con el problema que este proyecto intenta resolver: la logística. La mayoría de pacientes son personas mayores, muchas veces con movilidad reducida y que viven en zonas rurales o lejos de los hospitales. Para ellos, desplazarse a diario a un centro de rehabilitación supone un coste y un esfuerzo enorme, tanto personal como familiar.

Esto, sumado a que los servicios de rehabilitación suelen estar saturados, hace que los pacientes no reciban toda la terapia que necesitan. Por eso urge buscar soluciones tecnológicas que lleven la rehabilitación del hospital a casa, permitiendo al paciente ejercitarse sin perder la supervisión de su médico.

1.1. Contexto del Proyecto

Este Trabajo de Fin de Grado no es una iniciativa aislada, sino que forma parte de una línea de investigación académica de la Universidad de Burgos. El propósito de esta investigación es estudiar cómo la tecnología móvil puede mejorar la calidad de vida en enfermedades neurodegenerativas.

Dentro de este marco, mi trabajo se centra en el reto de la telerrehabilitación. El objetivo de este TFG ha sido construir la infraestructura software (la aplicación y el sistema de datos) necesaria para digitalizar los movimientos del paciente. Esta herramienta servirá de base tecnológica para que, en el futuro, otros proyectos puedan realizar análisis de movimiento y corrección postural.

1.2. Motivación y Objetivos Generales

La motivación principal del proyecto es eliminar la barrera de la distancia. Queremos usar la tecnología para crear un puente entre el terapeuta y el paciente, permitiendo que la terapia continúe, aunque no haya una visita presencial.

El objetivo general ha sido diseñar y desarrollar una aplicación móvil nativa completa que cumpla dos funciones clave:

- Para el Paciente (Accesibilidad): Ofrecer una herramienta para hacer ejercicios en casa siguiendo vídeos de ejemplo. Dado que los usuarios tienen dificultades motoras y no suelen ser expertos tecnológicos, la prioridad ha sido crear una interfaz extremadamente sencilla, accesible y fácil de usar.
- Para el Terapeuta (Control): Proveer una herramienta de gestión para asignar tareas y, lo más importante, verificar el trabajo. La aplicación permite al terapeuta ver los vídeos grabados por el paciente para evaluar su evolución de forma diferida (asíncrona), optimizando su tiempo de consulta.

1.3. Entregables y Repositorio

El resultado tangible de este trabajo es un prototipo funcional de aplicación Android.

Esta solución integra los roles de Administrador, Terapeuta y Paciente, cubriendo todo el ciclo: desde que se manda el ejercicio hasta que el paciente se graba y el médico lo revisa. La aplicación incluye internamente toda la lógica necesaria (base de datos y servidor) para funcionar de manera robusta.

Para facilitar la evaluación técnica del trabajo, todo el código fuente generado y la documentación del proyecto están disponibles en el siguiente repositorio de GitHub:

https://github.com/andrespuentesg/TFG-GII24.36-AndresPuentes

2. Objetivos del Proyecto

El propósito de este Trabajo de Fin de Grado trasciende el mero desarrollo de software; busca aportar una solución tecnológica a una problemática sanitaria real. Para estructurar el alcance del proyecto, se establecieron tres tipos de objetivos diferenciados al inicio del desarrollo: funcionales, técnicos y personales.

2.1. Objetivos Funcionales

Estos objetivos describen qué debe hacer la aplicación para ser útil tanto para los pacientes como para los terapeutas.

- Digitalización de la rehabilitación: El objetivo principal es sustituir el papel por una herramienta digital que cubra todo el proceso:
    - Asignación: Que el terapeuta pueda crear rutinas específicas para cada día desde su panel.
    - Realización: Que el paciente vea claramente qué le toca hacer hoy y tenga un vídeo de ejemplo para guiarse.
    - Revisión: Que el terapeuta pueda ver los vídeos grabados por el paciente y ponerles una nota.
- Accesibilidad Real (Diseño Adaptado): Sabiendo que los usuarios tienen Parkinson (temblores) y suelen ser mayores, la interfaz no puede ser la típica de Android. El objetivo es crear pantallas con botones muy grandes, textos claros y navegación sencilla, evitando menús complejos para que cualquiera pueda usarla sin frustrarse.
- Grabación Obligatoria de Ejercicios: Para dar una sesión por válida, el sistema debe obligar a grabar el ejercicio. No sirve con marcar una casilla de "hecho". Necesitamos el vídeo por dos razones: para que el médico verifique que el movimiento es correcto y para tener datos reales que sirvan al proyecto de investigación.
- Sistema de Roles: Que la aplicación se comporte de forma distinta según quién entre: el Administrador gestiona usuarios, el Terapeuta gestiona terapias y el Paciente solo ve sus ejercicios.

2.2. Objetivos Técnicos

Aquí se detallan los retos de programación y las herramientas elegidas para construir el sistema.

- Arquitectura MVVM (Modelo-Vista-ViewModel): No programar todo el código "a lo bruto" en una sola clase. El objetivo es separar la lógica (cálculos, base de datos) de la parte visual (pantallas). Esto hace que el código sea más limpio, fácil de arreglar si falla y más profesional.
- Uso de Jetpack Compose: Aprender y utilizar la nueva forma de crear interfaces en Android. El objetivo era dejar atrás el sistema antiguo de XML y dominar la

tecnología que piden hoy en día en las empresas, basada en código Kotlin declarativo.

- Integración de Cámara (CameraX): Uno de los puntos más complejos. El objetivo no es llamar a la cámara del móvil, sino integrar la cámara dentro de nuestra pantalla. Esto implica gestionar que no se bloquee, que grabe el audio y vídeo correctamente y que el archivo se guarde bien en la memoria interna del dispositivo.
- Gestión de Datos y Almacenamiento: Crear un sistema para guardar los vídeos en local de forma segura (para que no se pierdan) y diseñar una base de datos que relacione correctamente a cada paciente con sus ejercicios y su terapeuta.

2.3. Objetivos Personales

Más allá de aprobar el TFG, este proyecto tenía metas de aprendizaje propio:

- Aprendizaje "Full Stack" en Móvil: Enfrentarme al ciclo completo de desarrollo. Tanto la parte de hacer la interfaz tanto entender cómo funciona todo por detrás: crear mi propia API, diseñar la base de datos y conectarlo todo.
- Resolver problemas reales: Salir de los ejercicios de clase y enfrentarme a problemas que tienen soluciones más complejas y que me hagan salir de mi zona de confort y tratar de expandir mis conocimientos, como los fallos de la cámara en ciertos dispositivos o cómo manejar archivos de vídeo pesados sin que la app se cierre.
- Acercamiento al sector e-Health: Aprender a tomar requisitos no de un informático, sino de necesidades médicas reales, entendiendo que en una app de salud la prioridad absoluta es que el paciente sea capaz de usarla sin ayuda.

3. CONCEPTOS TEÓRICOS

(AQUÍ ESTOY BASTANTE PERDIDO Y NO SE MUY BIEN DE QUE CONSTA ESTE APARTADO NI QUE PODRIA PONER AQUÍ)

4. Técnicas y Herramientas

En este capítulo se describen las metodologías, patrones de diseño y tecnologías seleccionadas para la construcción del sistema. La elección de este stack tecnológico no es arbitraria, sino que responde a la necesidad de crear un software moderno, mantenible y adaptado a los estándares actuales de desarrollo móvil.

4.1. Metodología de Gestión: Marco de Trabajo Scrum

Para la gestión y organización del ciclo de vida del desarrollo, se ha optado por Scrum. Este marco de trabajo ágil está especialmente diseñado para entornos donde los requisitos son cambiantes o poco definidos al inicio, permitiendo abordar la complejidad del proyecto mediante un enfoque iterativo e incremental.

A diferencia de las metodologías tradicionales en cascada (Waterfall), donde se planifica todo al principio, Scrum ha permitido una entrega de valor continua. El desarrollo se ha dividido en ciclos temporales cortos y de duración fija denominados Sprints, lo que ha facilitado la incorporación temprana de feedback y la detección rápida de errores técnicos o de diseño.

Dado que este TFG es un proyecto unipersonal, se ha realizado una adaptación académica de la metodología, ajustando los roles y eventos estándar a la realidad del trabajo, pero manteniendo intactos sus tres pilares fundamentales: transparencia en el progreso, inspección constante del código y adaptación ante los imprevistos técnicos.

4.1.1. Roles y Responsabilidades

Aunque Scrum define roles para equipos multidisciplinares, en este proyecto se han distribuido las responsabilidades de la siguiente manera:

- Product Owner (Tutores y Requisitos Médicos): Este rol ha sido asumido de forma colaborativa. Los tutores académicos, actuando como representantes de las necesidades clínicas ("el cliente"), han sido los encargados de validar la utilidad de las funciones y priorizar los requisitos (por ejemplo, decidir qué datos debe ver el terapeuta o cómo debe ser la interfaz del paciente).
- Scrum Master y Equipo de Desarrollo (El Alumno): Como desarrollador único, he asumido la responsabilidad técnica total. Esto ha implicado no solo la escritura de código (Dev Team), sino también la autogestión del proceso (Scrum Master): eliminar bloqueos técnicos (investigación de librerías), asegurar el cumplimiento de los plazos y mantener la calidad del software.

4.1.2. Artefactos de Gestión

Para mantener el orden y la trazabilidad del trabajo, se han generado los siguientes artefactos documentales:

- Product Backlog (Pila del Producto): Un inventario vivo y ordenado de todas las funcionalidades, mejoras y correcciones necesarias. Este listado no fue estático, sino que evolucionó a medida que se descubrían nuevas necesidades técnicas (como la necesidad de crear un reproductor propio en lugar de usar uno externo).

- Sprint Backlog: El subconjunto de tareas seleccionadas para ejecutar en un periodo concreto. Al inicio de cada semana, se seleccionaban los ítems más prioritarios del Product Backlog para transformarlos en trabajo real.
- Incremento de Software: Al finalizar cada ciclo, el resultado no era documentación, sino una versión funcional de la aplicación. Esto permitía probar la app en el dispositivo real continuamente, asegurando que las nuevas funciones (ej: grabación de vídeo) se integraban correctamente con las anteriores (ej: login de usuario).

4.1.3. Eventos y Ceremonias

El ritmo de trabajo se estructuró en torno a Sprints semanales, marcados por los siguientes hitos:

- Sprint Planning (Planificación): Sesión de análisis personal al inicio de la semana para estimar la dificultad de las tareas (ej: "¿Cuánto tardaré en integrar la API de cámara?") y definir el objetivo del Sprint.
- Sprint Review (Revisión con Tutores): Las reuniones periódicas de seguimiento actuaron como Sprint Review. En ellas se presentaba el incremento funcional (la app funcionando), se recibía retroalimentación directa y se validaba si el camino tomado era el correcto.
- Sprint Retrospective (Mejora Continua): Tras cada entrega, se realizaba un análisis técnico interno para identificar qué causó problemas (ej: "La librería X es inestable") y cómo mejorar el proceso para el siguiente ciclo.

4.2. Fundamentos de Arquitectura e Ingeniería

La calidad de un proyecto software no se mide solo por su funcionamiento final, sino por su estructura interna. Para asegurar que la aplicación sea robusta, escalable y fácil de mantener, se han cimentado las bases del desarrollo sobre tres pilares de la ingeniería de software moderna en Android.

4.2.1. Patrón de Arquitectura: MVVM (Model-View-ViewModel)

Para evitar la creación de clases monolíticas y difíciles de mantener, se ha implementado el patrón de arquitectura MVVM. Este diseño tiene como objetivo principal la separación de responsabilidades, desacoplando totalmente la interfaz gráfica de los datos y las reglas internas de funcionamiento.

La estructura se divide en tres capas claramente diferenciadas:

- Modelo (Model): Es la capa encargada de la gestión de datos. Su responsabilidad es abstraer el origen de la información, decidiendo si los datos deben obtenerse de la base de datos local (persistencia) o solicitarse a la API remota (red). De esta forma, el resto de la aplicación no necesita saber de dónde vienen los datos, solo consumirlos.
- Vista (View): Representa la capa visual. En este proyecto, implementada con Jetpack Compose, es un componente "pasivo" e inteligente. Su única función es pintar en pantalla el estado actual que recibe y capturar las interacciones del

usuario (toques, gestos) para enviarlas hacia arriba, sin tomar decisiones por sí misma.

- ViewModel: Actúa como el gestor de estado. Es el intermediario que almacena los datos necesarios para la interfaz y procesa las reglas de la aplicación. Expone flujos de información observables a los que la Vista se suscribe. Lo crucial de este componente es que no tiene referencia a la Vista, lo que permite realizar pruebas unitarias sobre él sin necesidad de un emulador o interfaz gráfica.

4.2.2. Paradigma de Interfaz Declarativa (UI)

Se ha tomado la decisión técnica de abandonar el paradigma imperativo clásico (basado en XML) para adoptar la programación declarativa.

A diferencia del modelo tradicional, donde el desarrollador debía instruir manualmente a la interfaz sobre cómo cambiar (ej: "busca el botón y cambia su color a rojo"), en el modelo declarativo se describe qué debe aparecer en pantalla en función del estado actual de la aplicación.

- La UI como función del estado: La interfaz se concibe como un reflejo directo de los datos. Si el estado cambia (por ejemplo, pasa de "Cargando" a "Lista de Pacientes"), el sistema se encarga automáticamente de la recomposición: destruir y volver a dibujar solo los componentes necesarios para reflejar el nuevo dato.
- Ventajas en el proyecto: Este enfoque elimina una categoría entera de errores comunes de sincronización entre los datos y la vista, simplificando drásticamente el código necesario para gestionar pantallas complejas y dinámicas.

4.2.3. Modelo de Ejecución Asíncrona

Dado que la aplicación realiza operaciones intensivas —como la escritura de vídeo en disco, el procesamiento de imágenes de la cámara o la comunicación con el servidor—, ha sido crítico implementar un modelo de concurrencia estructurada.

El objetivo es evitar el bloqueo del Hilo Principal (Main Thread), que es el encargado de dibujar la interfaz a 60 fotogramas por segundo. Si este hilo se ocupa con una tarea pesada, la aplicación se congela y el sistema operativo fuerza el cierre (error ANR).

Para solucionarlo, se derivan estas tareas a hilos secundarios (background threads) de manera eficiente. Esto permite que la aplicación siga respondiendo fluidamente a los toques del usuario mientras, en segundo plano, se procesan los archivos o se realizan las validaciones de seguridad, notificando el resultado a la interfaz únicamente cuando la tarea ha finalizado.

4.3. Entorno y Lenguajes de Desarrollo

La elección del stack tecnológico se ha basado en la premisa de utilizar herramientas nativas y modernas que garanticen el rendimiento y la sostenibilidad del código a largo plazo.

Android Studio

Se ha utilizado el Entorno de Desarrollo Integrado (IDE) oficial de Google como centro de mando del proyecto. Basado en la potencia de IntelliJ IDEA, esta herramienta ha sido fundamental por tres capacidades críticas para el desarrollo:

- Gestión del Ciclo de Vida con Gradle: Automatiza la gestión de dependencias y la compilación, permitiendo integrar librerías complejas (como CameraX o Retrofit) de manera eficiente.
- Depuración Avanzada: Herramientas como el Logcat para el rastreo de errores en tiempo real y el Database Inspector, que ha permitido visualizar y corregir la base de datos interna sin necesidad de extraer archivos del dispositivo.
- Soporte para Compose: La funcionalidad de Live Preview ha permitido iterar el diseño de las pantallas visualizando los cambios en el código de forma instantánea, acelerando el desarrollo de la interfaz.

Kotlin

Se ha elegido Kotlin como lenguaje vehicular del proyecto, descartando Java. Es un lenguaje moderno, de tipado estático y totalmente interoperable, que se ha convertido en el estándar de la industria.

Justificación técnica:

- Seguridad (Null Safety): Su sistema de tipos está diseñado para eliminar el riesgo de NullPointerException, uno de los errores más comunes y críticos que provocan el cierre inesperado de aplicaciones. Esto es vital en una app de salud donde la estabilidad es prioritaria.
- Expresividad: Permite escribir código más conciso y legible. Características como las Data Classes (para los modelos de datos) o las Extension Functions han simplificado la estructura del código.
- Soporte de Concurrencia: Su integración nativa con Corrutinas ha sido la clave para gestionar las tareas asíncronas (grabación, red) de forma sencilla y eficiente.

Jetpack Compose

Para la construcción de la interfaz de usuario se ha adoptado Jetpack Compose, el toolkit moderno de Android. Esta decisión supone un cambio radical respecto al sistema clásico de Vistas (XML).

- Paradigma Declarativo: Permite describir la interfaz íntegramente en Kotlin. Esto elimina la fragmentación entre el código lógico y el diseño visual, reduciendo la complejidad del proyecto.
- Componentización: Facilita la creación de elementos visuales pequeños y reutilizables (como botones personalizados o tarjetas de paciente).
- Gestión de Estado: Simplifica la sincronización entre los datos y la pantalla. Al ser reactivo, la interfaz se actualiza automáticamente cuando cambian los datos del ViewModel, evitando errores de inconsistencia visual.

Material Design 3

Dada la importancia de la accesibilidad en este TFG, se ha implementado la última versión del sistema de diseño de Google. No se trata solo de una decisión estética, sino funcional.

- Accesibilidad por defecto: Sus componentes (botones, tarjetas, campos de texto) cumplen con los estándares de usabilidad, garantizando áreas táctiles mínimas y contrastes adecuados.
- Consistencia Visual: Proporciona un sistema estandarizado de tipografías, espaciados y colores. Esto ha permitido crear una experiencia de usuario coherente y predecible, algo fundamental para reducir la carga cognitiva en pacientes mayores o con dificultades tecnológicas.

4.4. Librerías y APIs Especializadas

La complejidad funcional de la aplicación, especialmente en lo referido al manejo multimedia, ha requerido la integración de bibliotecas especializadas que abstraen la dificultad del hardware subyacente.

CameraX

Para el módulo crítico de grabación se ha implementado CameraX, parte de la suite Android Jetpack. La elección de esta librería frente a las APIs antiguas (Camera1/Camera2) responde a la necesidad de estabilidad frente a la fragmentación del ecosistema Android.

- Ciclo de vida (Lifecycle-aware): Su mayor virtud es la gestión automática de recursos. CameraX se vincula al ciclo de vida de la actividad, abriendo y cerrando la cámara automáticamente cuando el usuario entra o sale de la pantalla. Esto previene errores comunes de "Cámara no disponible" y bloqueos de hardware, garantizando que el dispositivo esté siempre listo para grabar la terapia.
- Compatibilidad: Asegura un comportamiento consistente en una amplia gama de dispositivos, independientemente del fabricante.

ExoPlayer (Media3)

Para la reproducción de vídeo se ha descartado el componente nativo MediaPlayer debido a sus limitaciones de personalización y estabilidad. En su lugar, se ha integrado ExoPlayer, el reproductor de código abierto desarrollado por Google.

- Robustez: Ofrece un control preciso sobre el buffering y la carga de medios, fundamental para reproducir fluidamente tanto los vídeos de ejemplo (assets) como las grabaciones locales de los pacientes, que pueden tener tamaños considerables.
- Personalización: Ha permitido construir una interfaz de reproducción propia, adaptada a la accesibilidad del proyecto (botones grandes, controles simplificados), integrándose perfectamente en la UI de Compose.

Kotlin Coroutines

La gestión de la concurrencia es vital para mantener la fluidez de la aplicación (60 fps). Se han utilizado Corrutinas para simplificar la programación asíncrona.

- Concurrencia Estructurada: Mediante el uso de suspending functions y Scopes, se pueden ejecutar tareas pesadas (como procesar el vídeo tras la grabación o realizar consultas complejas a la base de datos) en hilos secundarios de forma secuencial y legible. Esto evita el "callback hell" y asegura que, si el usuario abandona una pantalla, las tareas en segundo plano se cancelen ordenadamente para no desperdiciar batería ni memoria.

Dibujo Vectorial (Canvas)

Para ciertos componentes visuales de la interfaz que requerían una personalización más allá de los estándares de Material Design, se ha recurrido al dibujo programático mediante Canvas.

- Visualización Dinámica: Esta técnica permite renderizar gráficos 2D utilizando primitivas geométricas y matemáticas en lugar de imágenes estáticas. Esto garantiza que los elementos visuales escalen perfectamente a cualquier resolución de pantalla (desde teléfonos pequeños hasta tablets grandes) sin perder nitidez, ofreciendo un control total sobre cada píxel renderizado.

4.5. Persistencia y Seguridad

Dada la naturaleza sensible de los datos tratados (imágenes de pacientes y registros de salud), la arquitectura de almacenamiento y seguridad ha sido prioritaria.

Android File Storage API (Almacenamiento Interno)

Para la persistencia física de los archivos de vídeo se ha implementado una estrategia estricta de privacidad utilizando el Almacenamiento Interno.

- Aislamiento: Se ha optado por guardar los vídeos en el directorio privado de la aplicación. A diferencia del almacenamiento externo o la galería pública, este espacio funciona como una "caja fuerte": el sistema operativo impide que otras aplicaciones instaladas o usuarios externos accedan a estos archivos. Esto garantiza que las grabaciones de las sesiones médicas permanezcan confidenciales y seguras por diseño.

Android Permissions API

El acceso al hardware se gestiona mediante el sistema de permisos en tiempo de ejecución (Runtime Permissions).

- Control del Usuario: La aplicación no asume privilegios; solicita explícitamente autorización para el uso de la Cámara y el Micrófono en el momento justo en que son necesarios (y no antes).
- Manejo de Estados: Se ha programado la lógica necesaria para gestionar situaciones donde el usuario deniega el permiso, degradando la funcionalidad de

la app de forma elegante y explicándole la necesidad de dichos permisos para la terapia, cumpliendo con las mejores prácticas de privacidad de Android.

4.6. Control de Versiones

Git y GitHub

La gestión del código fuente y el control de cambios se han profesionalizado mediante el uso de Git.

- Trazabilidad: Este sistema ha permitido mantener un historial detallado de cada modificación, facilitando la experimentación con nuevas funcionalidades (mediante el uso de ramas o branches) sin poner en riesgo la versión estable de la aplicación.
- Repositorio Remoto: Se ha utilizado GitHub como plataforma de alojamiento en la nube. Más allá de servir como copia de seguridad distribuida, ha permitido organizar el ciclo de vida del desarrollo y documentar la evolución del proyecto, sirviendo como evidencia del trabajo realizado ante el tribunal académico.

5 ASPECTOS RELEVANTES

5.1. El punto de partida: Motivación social y desafío técnico

El desarrollo de este proyecto se inició con la fase de selección dentro de la oferta de propuestas de la Universidad de Burgos. La decisión de abordar este TFG no fue arbitraria, sino que respondió a una decisión estratégica fundamentada en la convergencia de dos motivaciones principales que han actuado como motor de todo el proceso.

1. Impacto Social: Ingeniería con Propósito

En primer lugar, existía una fuerte vocación por la utilidad práctica. El proyecto planteaba una solución tecnológica a un problema tangible y doloroso: las barreras de acceso a la rehabilitación en pacientes con Parkinson. El factor determinante fue la posibilidad de trascender el ejercicio académico puro; no se trataba únicamente de escribir líneas de código para aprobar una asignatura, sino de construir una herramienta de e-Health con potencial de utilidad pública. La oportunidad de aportar una mejora, por pequeña que fuera, a la calidad de vida de un colectivo vulnerable y a menudo dependiente, dotó al trabajo de un sentido de responsabilidad que exigió un nivel de calidad superior al de un proyecto estándar.

2. El Desafío Profesional: Salto al Desarrollo Nativo En segundo lugar, el proyecto representaba el escenario ideal para el crecimiento profesional. Se identificó este TFG como la oportunidad perfecta para realizar la transición de "estudiante" a "ingeniero de software". Al tratarse de una aplicación completa (Full App) —que abarca desde la gestión de bases de datos y usuarios hasta la manipulación compleja de hardware multimedia—, el proyecto obligaba a salir de la zona de confort. La complejidad de los requisitos implicaba enfrentarse de forma autónoma a problemas de programación no triviales, fomentando una dinámica de autoaprendizaje para dominar tecnologías punteras en el mercado laboral actual que no se habían profundizado durante el grado.

5.2. Estrategia Tecnológica: La apuesta por el desarrollo nativo

De forma paralela a la definición de requisitos, se abordó una de las decisiones arquitectónicas más trascendentes: la elección del ecosistema de desarrollo. Si bien se evaluó la viabilidad de soluciones multiplataforma o híbridas (como Flutter o React Native) para agilizar el despliegue, finalmente se optó por una estrategia de Desarrollo Nativo puro utilizando el SDK de Android y Kotlin.

Esta decisión no fue arbitraria, sino que se fundamentó en cuatro pilares técnicos alineados con las necesidades críticas del sistema:

- Acceso Directo al Hardware (Sin Intermediarios): Dado que el núcleo del proyecto reside en la captura intensiva de vídeo (CameraX) y el almacenamiento en disco, la prioridad era minimizar la latencia. Las soluciones híbridas introducen "puentes" o capas de abstracción que suelen complicar el ciclo de vida del hardware. El desarrollo nativo ofrece la integración más directa y eficiente posible con la cámara, garantizando estabilidad y rendimiento en la grabación.
- Modernidad y Robustez (Kotlin First): Se eligió Kotlin sobre Java por ser el lenguaje estándar actual ("First-class citizen") recomendado por Google. Más allá de su sintaxis concisa, su sistema de tipos con Null Safety (seguridad ante nulos) permite eliminar una categoría entera de errores comunes en tiempo de ejecución, resultando en una aplicación más estable y un código más mantenible.
- Potencia del Ecosistema (Tooling): La elección de Android Studio como IDE proporcionó el entorno más completo para la ingeniería de software. Herramientas avanzadas como el Database Inspector (para ver la base de datos en tiempo real), el Layout Inspector o el Profiler de rendimiento fueron indispensables para depurar una aplicación compleja que gestiona hilos en segundo plano y multimedia.
- Optimización de Recursos y Curva de Aprendizaje: Desde el punto de vista de la gestión del proyecto, se decidió capitalizar la base de conocimientos adquirida durante el grado. Aprovechar la experiencia previa en Android permitió reducir la curva de aprendizaje inicial. Esto fue una decisión estratégica: ahorró tiempo en los fundamentos para poder invertir ese esfuerzo de autoformación en dominar las tecnologías avanzadas y específicas que requería el TFG (como Jetpack Compose o Corrutinas).

5.3. Ingeniería de Requisitos: Un enfoque evolutivo

Simultáneamente a la definición de la arquitectura técnica, se desplegó la fase de análisis y diseño. Dada la naturaleza de investigación del proyecto, se descartó un modelo en cascada (donde todos los requisitos se cierran al inicio) en favor de una estrategia de refinamiento incremental.

El alcance del sistema no fue estático, sino que maduró a través de ciclos de retroalimentación continuos. Las reuniones semanales con los tutores funcionaron como sesiones de validación, permitiendo transformar ideas abstractas en especificaciones técnicas concretas mediante un proceso iterativo:

- Requisitos Funcionales: Se estructuró el sistema en torno a tres actores clave: Administrador, Terapeuta y Paciente. El trabajo de análisis se centró en definir los flujos de información entre estos roles, priorizando los casos de uso nucleares: la gestión segura de identidades, la lógica de asignación de tratamientos y, fundamentalmente, el ciclo de vida de la grabación (captura, almacenamiento y sincronización).
- Requisitos No Funcionales: Se identificaron atributos de calidad que condicionarían toda la implementación.
    - Usabilidad Crítica: Debido a la sintomatología motora del usuario final (Parkinson), la accesibilidad no se trató como una característica deseable, sino como un requisito de viabilidad. Esto obligó a diseñar interfaces con tolerancia al error y alta legibilidad.
    - Robustez: Se estableció que la aplicación debía ser resiliente a fallos, garantizando que una interrupción durante la grabación nunca resultara en la pérdida de datos clínicos.

Esta inversión temprana en el diseño fue una decisión estratégica para la calidad del código. Al dedicar esfuerzo a definir con precisión las reglas de negocio antes de la implementación, se logró una fase de desarrollo mucho más fluida, minimizando la incertidumbre y evitando la deuda técnica asociada a la reescritura de componentes mal planificados.

5.4. Ingeniería de Datos: Diseño e Implementación

Tras la consolidación de los requisitos, se procedió a cimentar la base del sistema. Lejos de iniciar la programación de forma precipitada, se priorizó el análisis y estructuración de la información, entendiendo que un modelo de datos robusto es prerrequisito para cualquier lógica de negocio fiable.

El proceso se ejecutó en dos fases secuenciales:

- Fase 1: Modelado Conceptual y Lógico: Se elaboró el diseño teórico utilizando diagramas Entidad-Relación (E-R). Durante esta etapa, el esfuerzo se centró en identificar las entidades y definir las relaciones (cardinalidades) de forma estricta. Se aplicaron reglas de normalización para evitar redundancias y se definieron las restricciones de integridad referencial "sobre el plano", asegurando la coherencia de los datos antes de su implementación técnica.
- Fase 2: Implementación Física: Una vez validado el esquema lógico, se procedió a su materialización en el Sistema Gestor de Bases de Datos. Se tradujo el diseño a sentencias SQL, creando las tablas, claves foráneas e índices necesarios para soportar las consultas de la aplicación.

Elección del Motor: PostgreSQL

Para la persistencia de los datos se seleccionó PostgreSQL. Esta decisión técnica se fundamentó en dos criterios estratégicos:

- Fiabilidad Crítica (Conformidad ACID): Al tratarse de un sistema que gestiona información de salud, la integridad de los datos no es negociable. PostgreSQL destaca por su robustez y su estricto cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). Esto garantiza que, ante fallos del sistema o errores en una transacción, la base de datos nunca quede en un estado inconsistente, algo vital para un historial médico.
- Optimización de la Curva de Aprendizaje: Desde el punto de vista de la gestión de recursos del proyecto, se valoró la experiencia previa adquirida con este motor en el grado. Utilizar una tecnología consolidada permitió agilizar la fase de infraestructura, minimizando el tiempo dedicado a la configuración del entorno para poder invertirlo en la lógica compleja de la aplicación móvil.

5.5. El salto al Backend: Desarrollo de la API REST

Una de las etapas más trascendentales del proyecto fue el desarrollo de la infraestructura de servidor. Este paso supuso trascender el desarrollo móvil para abordar la ingeniería del Backend, enfrentando el desafío de conectar de forma segura la base de datos (PostgreSQL) con la aplicación Android.

Partiendo de la necesidad de centralizar el procesamiento y los datos, fue necesario diseñar e implementar una API REST (Interfaz de Programación de Aplicaciones) que actuase como puente de comunicación, garantizando que el intercambio de información fuera estandarizado y seguro.

5.5.1. Elección estratégica: FastAPI

Para la construcción de este componente crítico, la selección tecnológica se realizó bajo criterios de eficiencia y curva de aprendizaje. Se optó por FastAPI, un framework moderno de alto rendimiento para Python.

Esta decisión se fundamentó en dos pilares:

- Apalancamiento del conocimiento: Aunque la arquitectura web era un terreno nuevo, se contaba con un sólido dominio previo del lenguaje Python. Utilizar un framework basado en un lenguaje ya conocido permitió una estrategia de aprendizaje focalizado: el esfuerzo cognitivo se centró en entender los conceptos de arquitectura REST (rutas, verbos, serialización), eliminando la barrera de entrada de aprender una nueva sintaxis.
- Estándares de Documentación: FastAPI destacó por su capacidad de generar automáticamente documentación interactiva (basada en estándares Swagger/OpenAPI). Esto resultó vital para el desarrollo: permitía visualizar y probar los endpoints en tiempo real sin necesidad de herramientas externas, agilizando drásticamente la fase de depuración.

5.5.2. Implementación de las Reglas de Dominio

La construcción de los endpoints implicó una inmersión profunda en el protocolo HTTP, mapeando correctamente las operaciones del sistema a los verbos estándar (GET para consultas, POST para creaciones, etc.) y gestionando los códigos de estado adecuados.

El objetivo fue encapsular las reglas de funcionamiento del sistema en el servidor. La API no se limitó a ser una simple "pasarela de datos", sino que se programó para actuar como guardián de la integridad de la aplicación:

- Filtrado Contextual: Se implementaron filtros estrictos para asegurar el aislamiento de la información. Por ejemplo, el sistema garantiza matemáticamente que cuando un paciente solicita "sus ejercicios", la API nunca devuelva datos pertenecientes a otro usuario.
- Gestión de Estados: Control estricto de la asignación de tareas, impidiendo inconsistencias como la duplicación de rutinas en un mismo día o la modificación de historiales clínicos ya cerrados.

5.5.3. Seguridad y Aseguramiento de la Calidad (QA)

Dada la naturaleza crítica de los datos médicos gestionados, la seguridad se diseñó desde la base (Security by Design), no como un añadido final.

- Cifrado de Credenciales: Se implementó el algoritmo Bcrypt para el hashing de contraseñas, asegurando que las credenciales de los usuarios nunca se almacenen en texto plano en la base de datos, protegiéndolas incluso ante un hipotético acceso no autorizado.

Finalmente, para certificar la robustez del sistema, se desarrolló una batería de tests automatizados. Estas pruebas de integración verifican continuamente que las restricciones de privacidad y las reglas internas se cumplen estrictamente, garantizando que el sistema es resiliente ante cambios en el código.

5.6. Desarrollo de la Aplicación: Arquitectura Modular por Roles

Una vez desplegada la API y consolidada la estructura de datos, se inició la fase de implementación de la aplicación móvil. Dado que el sistema debía satisfacer necesidades operativas radicalmente opuestas según el usuario (gestión vs. accesibilidad), se tomó la decisión arquitectónica de segregar la interfaz.

Técnicamente, esto implicó diseñar tres flujos de navegación independientes dentro de una misma aplicación monolítica. Dependiendo del token de autenticación recibido en el login, la app adapta su comportamiento y apariencia, funcionando en la práctica como tres herramientas especializadas integradas en un solo binario.

5.6.1. Rol de Administrador: Gestión de Identidades

La primera interfaz desarrollada fue la del Administrador. Aunque visualmente se optó por un diseño sobrio y funcional, este módulo constituye la columna vertebral de la seguridad del sistema.

- Enfoque: Prioridad absoluta a la integridad de los datos sobre la estética.
- Implementación Técnica: Se desarrollaron pantallas tipo CRUD (Create, Read, Update, Delete) para la administración de usuarios. El desafío técnico residió en asegurar la integridad referencial desde el cliente.
    - Validación: El sistema impide la creación de registros huérfanos. Por ejemplo, al dar de alta un paciente, la app valida contra la API que el terapeuta asignado existe y está activo antes de confirmar la operación, actuando como primera barrera de defensa ante la corrupción de datos.

5.6.2. Rol de Terapeuta: Supervisión y Productividad

El siguiente paso fue la interfaz para el personal sanitario. La complejidad aumentó significativamente, ya que el terapeuta requiere procesar grandes volúmenes de información visual de manera ágil.

- Enfoque: Eficiencia en la visualización de datos y Feedback asíncrono.
- Implementación Técnica:

    - Listas Dinámicas: Se utilizó el componente LazyColumn de Compose para renderizar listas de pacientes de forma eficiente en memoria, cargando los datos bajo demanda.
    - Reproducción Multimedia: El componente crítico fue la integración de ExoPlayer. Se implementó una lógica de carga que permite al terapeuta visualizar los vídeos de las sesiones (descargados temporalmente desde la API o caché local) dentro de la propia app para asignar una calificación. Esto cierra el ciclo de rehabilitación, permitiendo la evaluación remota.

5.6.3. Rol de Paciente: Diseño Inclusivo y Accesible

Finalmente, se abordó la interfaz más crítica: la del paciente con Parkinson. Aquí, las guías de diseño estándar de Android se descartaron deliberadamente en favor de criterios de accesibilidad cognitiva y motora.

- Enfoque: Minimalismo, tolerancia al error y reducción de la carga cognitiva.
- Implementación Técnica:
    - Componentes Adaptados: Se diseñaron elementos de interfaz (UI) personalizados con áreas táctiles expandidas y alto contraste. Se eliminaron los menús de navegación jerárquicos, sustituyéndolos por un flujo lineal guiado.
    - Automatización (La "Sesión Guiada"): El mayor logro técnico fue la automatización del flujo de ejercicio. Se creó una pantalla donde la interacción física del usuario es casi nula: la aplicación toma el control ("Manos Libres"), reproduciendo el vídeo de ejemplo, iniciando cuentas atrás auditivas/visuales y gestionando el inicio y fin de la grabación de cámara automáticamente. Esto mitiga el problema de los temblores al interactuar con la pantalla.

5.6.4. Desafío de Ingeniería: Orquestación de Sesiones y Evolución

Más allá del diseño visual, la mayor dificultad técnica del proyecto residió en el desarrollo del "Motor de Sesiones": el conjunto de algoritmos internos que determinan qué ejercicios deben mostrarse, en qué orden y cuál es su estado actual.

Aunque el modelo teórico parecía sólido, la implementación práctica reveló limitaciones que obligaron a realizar un proceso de refactorización y evolución del software:

1. Evolución del Esquema de Datos y API Durante las pruebas de integración, se detectó que el esquema inicial era insuficiente para cubrir la casuística real del tratamiento. Fue necesario iterar sobre el diseño:

- Cambios en Backend: Se modificaron las tablas en PostgreSQL para incluir metadatos de control (banderas de estado, fechas de vigencia) y se desarrollaron nuevos endpoints en FastAPI para servir esta información granular.

- Lección aprendida: Esto demostró la importancia de contar con una arquitectura flexible que tolere cambios en el modelo de datos sin provocar una ruptura del sistema.

2. Gestión de Estados y Sincronización En el lado del cliente (Android), la coordinación entre lo que dice el servidor y lo que ve el usuario presentó dos retos de concurrencia:

- Filtrado Contextual: Inicialmente, existían discrepancias en la visualización de tareas. Se tuvo que depurar la lógica de consulta para asegurar un filtrado temporal estricto: la app debe discriminar entre sesiones pasadas, futuras y actuales, mostrando al paciente únicamente lo relevante para "el día de hoy" y ocultando el ruido.
- Consistencia del Estado "Completado": Uno de los bugs más complejos implicaba la falta de reactividad tras finalizar un ejercicio. El sistema permitía repetir tareas ya hechas debido a un retraso en la actualización del estado local.
    - Solución: Se implementó un patrón de actualización optimista con confirmación: al terminar la grabación, la app bloquea inmediatamente la tarea en la interfaz local mientras comunica el cambio a la base de datos en segundo plano, garantizando que el usuario percibe el sistema como instantáneo y consistente.

5.7. Ingeniería del Módulo Multimedia: Captura, Reproducción y Persistencia

Si bien las fases anteriores abordaron la lógica de gestión, la complejidad técnica del proyecto alcanzó su cota máxima en la ingeniería del módulo de vídeo. Este componente no se limitó a la simple grabación, sino que requirió orquestar un ciclo de vida multimedia completo: desde la reproducción fluida de guías visuales hasta la captura de evidencia clínica y su almacenamiento seguro, todo ello bajo restricciones críticas de accesibilidad y rendimiento.

5.7.1. Desarrollo de un Motor de Reproducción Nativo

En las fases iniciales de prototipado, se evaluó el uso de soluciones externas (WebViews con YouTube o Drive) para mostrar los ejercicios de ejemplo. Sin embargo, las pruebas de usabilidad desaconsejaron esta vía por introducir "fricción cognitiva": la aparición de publicidad, interfaces con controles diminutos y la salida del entorno seguro de la App generaban desorientación en el paciente.

Ante esta barrera de Experiencia de Usuario (UX), se tomó la decisión técnica de implementar un sistema de reproducción nativo (Embedded Player) basado en la librería Media3/ExoPlayer.

- Control Total del Ciclo de Vida: La ventaja diferencial no fue solo estética, sino funcional. Al tener control programático sobre el reproductor, se implementó un autómata de estados: el sistema detecta el evento STATE_ENDED (fin del vídeo) y desencadena automáticamente la transición a la pantalla de grabación.
- Resultado: Se eliminó la interacción manual ("Cerrar vídeo" -> "Abrir cámara"). El paciente es guiado por un flujo continuo donde la tecnología se vuelve invisible.

5.7.2. Estrategia de Persistencia: Abstracción y Escalabilidad

Uno de los dilemas arquitectónicos más relevantes fue la elección del soporte de almacenamiento. Se realizó un estudio de viabilidad para integrar servicios Cloud (AWS S3, Google Cloud Storage), pero las restricciones de presupuesto y tiempo aconsejaron una implementación local para el MVP (Producto Mínimo Viable).

No obstante, para no hipotecar el futuro del proyecto, se diseñó una Arquitectura "Cloud-Ready" mediante Abstracción:

- Implementación Local (Scoped Storage): Para garantizar la estabilidad y eliminar la latencia de red durante la defensa, los vídeos se guardan en el directorio privado del dispositivo.
- Desacoplamiento de Rutas: Lo innovador de la solución reside en el diseño del código. Se abstrajo la lógica de acceso a datos. La capa de presentación (la App) no sabe si el vídeo viene de una ruta local (file://) o remota (https://).
- Escalabilidad: Gracias a este diseño, la futura migración a la nube no requerirá reescribir la lógica de la aplicación; bastará con cambiar la implementación de la interfaz de repositorio para que apunte a un servidor remoto, manteniendo intacto el resto del sistema.

5.7.3. Gestión de Hardware y Verificación de Integridad

La integración de CameraX supuso enfrentarse a la inestabilidad inherente al hardware móvil. Durante las pruebas de estrés, se identificó un fallo crítico de corrupción de datos: ante interrupciones bruscas del sistema operativo, se generaban ocasionalmente archivos contenedores vacíos (0 bytes) o "corruptos" que provocaban el cierre inesperado (crash) del reproductor al intentar leerlos.

Para mitigar este riesgo, se desarrolló una capa de Programación Defensiva y Autocuración:

- Validación Post-Procesado: Antes de confirmar una sesión como "finalizada", el sistema ejecuta una rutina de verificación de integridad (Checksum funcional) sobre el archivo resultante.
- Gestión de Errores: Si el algoritmo detecta un tamaño nulo o una cabecera de vídeo ilegible, el sistema descarta el archivo corrupto de forma controlada y solicita al usuario una nueva toma o notifica el error, impidiendo que se persistan referencias a datos inválidos en la base de datos.

Esta combinación de reproducción integrada, arquitectura escalable y mecanismos de tolerancia a fallos convirtió una funcionalidad aparentemente estándar en el módulo más sofisticado y robusto del proyecto.

5.8. Balance Técnico

A modo de síntesis, es importante destacar que el desarrollo de este TFG ha trascendido la mera implementación de requisitos funcionales. Ha supuesto un ejercicio integral de ingeniería de software donde el valor principal reside en la toma de decisiones bajo incertidumbre.

Como se ha detallado a lo largo de este capítulo, el proyecto ha exigido superar obstáculos no contemplados en la teoría académica básica:

- La migración de paradigmas imperativos a reactivos (Jetpack Compose).
- La resolución de problemas de bajo nivel (hardware de cámara y sistemas de archivos).
- La gestión de la concurrencia y el estado en aplicaciones distribuidas (Cliente-Servidor).

La capacidad demostrada para refactorizar el modelo de datos ante nuevos hallazgos o implementar sistemas de autocuración para los archivos multimedia evidencia que el resultado final no es un prototipo estático, sino un sistema vivo, resiliente y construido sobre estándares profesionales de calidad.

6. TRABAJOS RELACIONADOS

7. Conclusiones

La finalización de este Trabajo de Fin de Grado representa la consecución de un hito tecnológico clave dentro de la línea de investigación propuesta por la Universidad de Burgos. Más allá del cumplimiento académico, el proyecto ha logrado diseñar, desarrollar y validar un sistema integral de telerrehabilitación que no solo satisface los requisitos funcionales iniciales, sino que aporta soluciones de ingeniería robustas a problemas complejos de accesibilidad y captura de datos multimedia.

A continuación, se detallan las conclusiones extraídas del desarrollo, clasificadas por áreas de impacto:

7.1. Conclusiones Funcionales y Clínicas

El sistema desarrollado ha validado la viabilidad técnica de trasladar la rehabilitación neurológica al entorno domiciliario sin perder rigor en el seguimiento.

- Cambio de Paradigma (Modelo Asíncrono): La aplicación rompe eficazmente la barrera geográfica y temporal. Al implementar un modelo de comunicación asíncrona (el terapeuta pauta cuando puede; el paciente ejecuta cuando quiere), se soluciona el cuello de botella de la saturación en consultas presenciales y se democratiza el acceso a la terapia para pacientes del entorno rural.
- Digitalización "End-to-End": Se ha logrado informatizar con éxito todo el flujo de trabajo: desde la prescripción de rutinas hasta la evaluación cuantitativa y cualitativa. Esto transforma la rehabilitación de un evento episódico (visitas puntuales) a un proceso continuo, trazable y medible.

7.2. Conclusiones sobre Accesibilidad y UX

Uno de los mayores retos de ingeniería fue adaptar la tecnología a un perfil de usuario con limitaciones motoras severas (bradicinesia, temblores) y baja alfabetización digital.

- Viabilidad del Diseño Inclusivo: El uso de Jetpack Compose ha permitido implementar una interfaz no convencional, alejada de los estándares densos de Android. La priorización de elementos de gran tamaño y altos contrastes ha validado la hipótesis de que una interfaz adaptada reduce drásticamente los errores de interacción ("falsos toques").
- Gestión de la Carga Cognitiva: La implementación de flujos guiados y automatizados (modo "Manos Libres") ha demostrado ser crítica. Al delegar en el software la gestión de los tiempos, la cuenta atrás y el control de la cámara, se libera al paciente de la fricción tecnológica, permitiéndole concentrarse exclusivamente en la ejecución del movimiento físico.

7.3. Conclusiones Técnicas

Desde la perspectiva de la Ingeniería del Software, el proyecto ha superado desafíos de bajo nivel para garantizar la estabilidad y la calidad del dato.

- Integridad en la Captura Multimedia: La integración de CameraX junto con los mecanismos de verificación de archivos (checksums) ha sido un éxito técnico.

Se han resuelto los problemas de corrupción de datos asociados a interrupciones del sistema operativo, garantizando que el vídeo —el activo más valioso del sistema— se genere y persista siempre de forma íntegra.

- Arquitectura Escalable (MVVM + Clean Architecture): La adopción estricta del patrón Modelo-Vista-ViewModel asegura un código modular y testable. Aunque el prototipo actual utiliza almacenamiento local, la clara separación de responsabilidades en la capa de datos (Repository Pattern) y la abstracción de fuentes de datos dejan al sistema preparado ("Cloud-Ready") para conectarse a un servidor remoto en el futuro con cambios mínimos en el código base.
- Eficiencia y Concurrencia: La aplicación ha demostrado un rendimiento fluido (60 fps) incluso bajo carga, gestionando tareas pesadas como la previsualización de cámara y la escritura de archivos en alta definición simultáneamente, gracias al uso eficiente de la programación asíncrona con Kotlin Coroutines.

7.4. Aportación a la Investigación (Valor del Dato)

Este TFG no se constituye como un producto aislado, sino como el motor de adquisición de datos para las siguientes fases de la investigación académica.

- Homogeneización del Dataset: Hasta ahora, la recolección de vídeos se enfrentaba a la variabilidad del entorno. Esta aplicación estandariza técnicamente el formato, la resolución y el encuadre de las grabaciones, proporcionando un dataset homogéneo y de alta calidad.
- Habilitador de IA: Esta estandarización es el prerrequisito indispensable para que los futuros algoritmos de Inteligencia Artificial (detección de poses y esqueletización) puedan entrenarse con fiabilidad. Sin la infraestructura de captura desarrollada en este TFG, la fase de análisis automático sería inviable.

7.5. Líneas Futuras de Trabajo

El desarrollo actual constituye una base sólida y funcional que cumple con los objetivos del TFG. Sin embargo, el proyecto se concibe como un sistema vivo con un amplio margen de recorrido. A continuación, se detallan las principales vías de evolución técnica identificadas para futuras iteraciones del software:

1. Migración a Arquitectura Distribuida (Cloud)

Actualmente, la persistencia de datos y el almacenamiento de los archivos de vídeo se realizan de forma local en el dispositivo (Edge Storage). El siguiente paso lógico en la hoja de ruta es la transición hacia una arquitectura cliente-servidor completa.

- Objetivo: Centralizar la información clínica en una infraestructura en la nube. Esto permitirá al terapeuta acceder a las grabaciones y realizar evaluaciones desde su estación de trabajo en el hospital mediante un panel web, eliminando la dependencia física de la tablet del paciente y facilitando la escalabilidad del servicio a cientos de usuarios simultáneos.

2. Tele-rehabilitación Síncrona (Control Remoto)

Se ha identificado un subgrupo de pacientes con deterioro cognitivo o afectación motora severa para los cuales la interacción táctil, por simplificada que sea, representa una barrera de entrada.

- Objetivo: Implementar un módulo de comunicación en tiempo real (utilizando tecnologías como WebSockets) que habilite un modo de "Sesión Asistida". Esto permitiría al terapeuta tomar el control remoto de la aplicación del paciente, lanzando los vídeos y gestionando la grabación desde su consola, reduciendo la interacción del usuario a la mera ejecución física del ejercicio. Esta funcionalidad fue valorada durante el análisis de requisitos, pero se priorizó la estabilidad del modelo asíncrono en esta primera fase.

3. Integración de Visión por Computador e IA

Esta línea representa la convergencia final con el proyecto de investigación macro. Consiste en trascender el almacenamiento de vídeo para pasar al procesamiento inteligente del mismo.

- Objetivo: Integrar librerías de Visión por Computador (como MediaPipe o ML Kit) para automatizar el análisis biomecánico. El sistema evolucionará para detectar métricas objetivas (velocidad, amplitud angular, frecuencia del temblor) y ofrecer feedback correctivo automático al paciente en tiempo real, sirviendo como herramienta de apoyo al diagnóstico cuantitativo para el facultativo.

4. Estrategias de Adherencia (Gamificación)

La rehabilitación en enfermedades neurodegenerativas exige una constancia diaria que frecuentemente genera fatiga psicológica y abandono del tratamiento.

- Objetivo: Implementar mecánicas de ludificación (Gamification) sobre el sistema de seguimiento actual. La introducción de sistemas de recompensas, medallas virtuales por constancia o rachas de días consecutivos busca impactar

positivamente en la psicología del paciente, aumentando la motivación intrínseca y mejorando las tasas de adherencia terapéutica a largo plazo.

5. Ciberseguridad y Cumplimiento Normativo (E2EE)

Vinculado a la migración a la nube (Punto 1), la protección del dato médico pasa de ser una característica local a un requisito crítico de transporte.

- Objetivo: Robustecer la capa de seguridad implementando protocolos de Cifrado de Extremo a Extremo (E2EE) para la transmisión de vídeo. Asimismo, se deberá auditar el sistema para garantizar el cumplimiento estricto del Reglamento General de Protección de Datos (RGPD) y las normativas sanitarias vigentes, asegurando que la información sensible viaje y se almacene bajo los más altos estándares de criptografía.